Karim Taha

Cairo,Egypt
+20-1115643549 kariem.taha2.7@gmail.com Karim Taha Kariiem

EDUCATION

Cairo University Faculty of Engineering **August 2018 – July 2023**
BSc. Computer Engineering, Grade: Excellent *Cairo, Egypt*

EXPERIENCE

Backend Engineer (Elixir / OTP, CQRS/ES) August 2025 - Present

Sonnen

- Designed and extended CQRS/Event-Sourcing workflows using Commanded: introduced new commands/events and updated projectors and process managers to support cross-service domain logic.
- Resolved a production aggregate state bug by redesigning event emission (two-event sequencing) to correctly handle timeouts and prevent incorrect state replacement involving Oban jobs.
- Re-architected JSON logging pipeline to align with OTP 26+ logger handler,filter,formatter model, adding structured metadata for Phoenix/Plug requests.
- Diagnosed and eliminated shutdown race conditions between dynamic supervisors and PubSub processes.
- Designed and implemented a reusable RabbitMQ client library with explicit producer/consumer behaviours and blocked-connection handling via Phoenix.PubSub state broadcasting.

Parse error recovery and incrementality for GHC June 2024 - Nov 2024

Haskell.org (GSoC 2024)

- GHC (Glasgow Haskell Compiler) is a state-of-the-art Haskell compiler, it's divided into several stages: parser, renamer, typechecker, desugarer and code gen.
- GHC's parser is not fault-tolerant, so any parsing error causes the whole compilation pipeline to fail, this affects the programming experience, since it leads to longer modify-compile-correct cycles.
- The project included two parts, the happy parser generator, and GHC's grammar file (happy user).
- A form of recovery, namely resumable parsing was implemented in happy parser generator. It had to be modified so upon encountring an error, it shifts a 'catch' token to the parsing stack, to determine where to resume parsing the grammar author should handle 'catch' explicitly in the grammar.
- The grammar file was modified to accept 'catch' token in positions where we want to resume parsing and where parsing can be recovered by the surrounding context.

Improving FreeBSD User Emulation of the Armv7 Architecture June 2023 - Sept 2023

QEMU.org (GSoC 2023)

- FreeBSD User is the name of the QEMU submodule for FreeBSD OS.
- To improve the Armv7 emulation, a set of syscalls needed to be implemented in FreeBSD User. Issuing a syscall in the guest OS works by translating the addresses of the input parameters from the QEMU address space to the host address space, call the corresponding syscall on the host, then copy the output parameters, if any, to the QEMU address space.
- The goal was to get a simple C++ hello-world program to compile and run inside an armv7 jail hosted on an amd64 machine using clang.

PROJECTS

Lang May 2023

- A compiler for a statically typed, expression based programming language, written in C.
- The implementation follows the literature standards, by dividing the compiler into a front-end and a back-end.
- The front-end is done using flex/bison parser generators to generate a parser in C.
- The back-end is divided into several stages: symbol tables constuction, type checking and code generation.

RSA Algorithm June 2022

- Simple implementation of RSA algorithm in Haskell packaged as a cabal library.
- Used the sockets api for communication between sender and receiver.
- Implemented chosen cipher text attack.

VHDL Pipelined-Processor **Jan 2022**

- A Harvard-architecture 5-stage pipelined processor, implemented in VHDL using Intel's ModelSim.
- The processor has a RISC-like instruction set architecture, with 8 word-sized general-purpose registers:R0-R7 and some special purpose registers: program counter(PC) And stack pointer(SP).
- The memory address space is 1 MB of 16-bit width and is word addressable. The data bus is 32 bits.

Search engine **June 2021**

- Search engine in Java, using Java's multi-threading facilities, namely thread pools.
- The Search engine is divided into two parts the web crawler and the indexer.
- The Web Crawler recursively downloads a specified list of web pages, recurring on each sub link.
- The Indexer reads every word, increases it's score then inserts the collected data into an SQL database, ready for query from a simple Web Gui.

TECHNICAL SKILLS

Languages: C/C++, Haskell, Python, javaScript/Typescript, SQL
Developer Tools: Emacs, Neovim, VS Code, GitHub
Technologies/Frameworks: FreeBSD, Linux, Git